

KatzLab Ureteroscope

Doing Everything on the Mint Box Alone

No Mac, no second machine, no Foxglove — one machine, start to finish

October 5, 2026

Why this guide is different from the main runbook

`docs/RUNBOOK.pdf` assumes two machines: the Mint box running the rig and Isaac Sim, and a Mac watching it remotely over Foxglove. That split exists only because Isaac Sim cannot run on macOS at all — it needs an NVIDIA GPU and Linux/Windows.

If you are sitting at the Mint box itself, none of that split is needed. Everything — the bridge, RViz, Isaac Sim, the real rig CAD, training — runs and is viewed locally, on one machine, with no WebSocket bridge, no second app, and no network setup. This guide is that simpler path.

Contents

1	Install ROS 2	2
1.1	Check the Mint/Ubuntu base	2
1.2	Run the setup script	2
1.3	Open a new terminal and verify	2
2	Sanity-check what needs nothing else first	2
3	Build the ROS 2 package	2
4	Bridge + RViz, all local	3
4.1	Send a test motion command	3
5	Isaac Sim: the real rig CAD	3
5.1	Look at it before driving it	3
5.2	Drive it live from the bridge	4
6	First training milestone: get the tip to the stone	4
6.1	Smoke-test	4
6.2	Install training dependencies	4
6.3	Train	4
7	Everything running at once — a reference layout	4
8	What to report back if something breaks	5

1 Install ROS 2

1.1 Check the Mint/Ubuntu base

The install script refuses to run on anything other than a Mint 22.x (Ubuntu 24.04 “noble”) base, on purpose:

```
cat /etc/upstream-release/lsb-release
```

You want `DISTRIB_CODENAME=noble`. If it says something else, stop and ask before continuing.

1.2 Run the setup script

From the repository root:

```
cd katzlab-ureteroscope
./ros2/setup-mint.sh
```

Installs ROS 2 Jazzy Desktop (RViz2 included), `colcon`, and `foxcglove-bridge` (unused in this guide, but harmless to have). A few GB, takes a while, safe to re-run if it fails partway.

1.3 Open a new terminal and verify

A terminal that was already open will not see the newly-installed ROS 2. Open a fresh one:

```
ros2 --version
which colcon
rviz2 --help | head -1
```

All three should print something, not “command not found.”

2 Sanity-check what needs nothing else first

These run right now, before ROS 2 or Isaac Sim are even touched. If either fails, fix it before going further — nothing downstream will work:

```
cd ros2/katzlab_bridge/test && python3 -m pytest -q
cd ../../isaac/training && python3 -m pytest test_reach_math.py -q
```

Expected: 6 passed, 1 skipped and 10 passed.

3 Build the ROS 2 package

```
cd ros2
colcon build --packages-select katzlab_bridge
source install/setup.bash
```

Do this in every new terminal you open for the rest of this guide (or add it to `~/.bashrc` once you are confident it works).

4 Bridge + RViz, all local

One command starts the bridge node and RViz together, no Foxglove, no WebSocket, nothing but this one machine:

```
ros2 launch katzlab_bridge sim.launch.py dry_run:=true
```

`dry_run:=true` means no Teensy/hardware needed yet — this exercises the ROS 2 wiring only. Drop the flag once hardware is attached.

An RViz window opens directly on this machine's own screen, showing the placeholder stick-figure model. Leave this terminal running.

4.1 Send a test motion command

From a second terminal:

```
source ~/katzlab-ureterscope/ros2/install/setup.bash

ros2 topic pub /ureterscope/cmd_velocity geometry_msgs/msg/Twist \
  "{linear: {x: 0.001}, angular: {z: 0.0, y: 0.0}}"
```

The model in RViz should move. Other useful checks from this same terminal:

```
ros2 topic echo /ureterscope/joint_states
ros2 service call /ureterscope/home std_srvs/srv/Trigger
ros2 service call /ureterscope/estop std_srvs/srv/Trigger
```

5 Isaac Sim: the real rig CAD

This is the part that is genuinely local-only no matter what — Isaac Sim cannot be watched remotely from a Mac at all, so being at this machine is the only way to see it, full stop.

5.1 Look at it before driving it

```
source /opt/ros/jazzy/setup.bash
~/isaacsim/python.sh ~/katzlab-ureterscope/ros2/isaac/preview_scene.py
```

A window opens with the real rig CAD (linear stage, rotation bearing, the actual Wolf RIWO ureterscope mesh) and the real 8-stone benchtop target grid. Check:

1. Does the scope's length look visually right relative to the table/linear stage? (There is a known open question about this — see `ros2/isaac/assets/ASSET-NOTES.md`.)
2. Do the 7 flexible-tip segments bend as a plausible continuous curve when driven through their range?
3. Do the 8 stone markers sit somewhere sensible near the tip?

If something looks visually wrong here, stop and note exactly what, before trusting anything built on top of it.

5.2 Drive it live from the bridge

With Section 4's `sim.launch.py` still running in its own terminal:

```
~/isaacsim/python.sh ~/katzlab-ureteroscope/ros2/isaac/load_rig.py
```

Re-send the test command from Section 4.1. Both the RViz window (placeholder model) and the Isaac Sim window (real CAD) should move together, within a step or two of each other.

6 First training milestone: get the tip to the stone

6.1 Smoke-test

```
~/isaacsim/python.sh \  
~/katzlab-ureteroscope/ros2/isaac/training/reach_task.py
```

Random actions, no training — confirms the environment runs end to end.

6.2 Install training dependencies

```
~/isaacsim/python.sh -m pip install gymnasium numpy stable-baselines3
```

6.3 Train

```
~/isaacsim/python.sh \  
~/katzlab-ureteroscope/ros2/isaac/training/train_reach.py \  
--total-timesteps 200000
```

State-based (tip/target *positions*, not camera images) on purpose — simpler problem first, and the prerequisite for a vision-based version later. Do not expect reliable stone-reaching from one run; watch for mean episode reward trending up and mean episode length trending down as the first sign anything is working.

7 Everything running at once — a reference layout

Four terminals (or four panes in one terminal multiplexer), all on the Mint box:

Terminal	Running
1	<code>ros2 launch katzlab_bridge sim.launch.py dry_run:=true</code>
2	<code>~/isaacsim/python.sh ../load_rig.py</code>
3	free, for <code>ros2 topic pub / service call</code> commands
4	free, for training once 1–3 are confirmed working

No Mac, no Foxglove, no second machine, no network configuration — one box, four terminals, everything visible on its own screen.

8 What to report back if something breaks

For each step: which numbered section, the exact command, and the full error output (not a paraphrase).

Section 5 (Isaac Sim) is the one most likely to need a fix on first contact with a real install — none of that code has been run against one yet. The two most likely failure points (the URDF importer extension’s exact module path, and Replicator/RL API field names) are both named in `ros2/isaac/README.md`.